E3: 245 Processor System Design — Laboratory Exercise 3

5-Stage Pipelined 32-bit RISC-V Processor with Direct-Mapped Instruction Cache

Anirudh Chaturvedi (SR No: 25850)
DESE, Indian Institute of Science, Bangalore

November 2026

1 Introduction & Design Specifications

This report documents the design, FPGA implementation, and post-route verification of a **5-stage pipelined 32-bit RISC-V (RV32I) processor** with a **direct-mapped Instruction Cache** targeting the Digilent Basys 3 FPGA board (`XC7A35T-1CPG236C`, Artix-7, speed grade `-1`).

- **Supported ISA (User Mode):** Basic RV32I arithmetic-logic instructions (R-type and I-type), `LUI`, branch instructions (`BEQ`, `BNE`, `BLT`, `BGE`, `BLTU`, `BGEU`), load/store instructions (`LB`, `LH`, `LW`, `LBU`, `LHU`, `SB`, `SH`, `SW`), and jump instructions (`JAL`, `JALR`). Atomic, `AUIPC`, and CSR instructions are omitted as specified.
- **Pipeline & Forwarding:** 5-stage pipeline (**IF, ID, EX, MEM, WB**) with full data forwarding from **EX/MEM** and **MEM/WB** outputs to the EX stage (`forwarding_unit.v`), plus **WB-to-ID** write-through bypass in the register file (`reg_file.v`).
- **Hazard Detection & Stall Unit (`hazard_unit.v`):** Detects load-use hazards (1-cycle stall of `PC` and `IF/ID`, bubble injected into `ID/EX`), resolves branches/jumps in EX (flushing `IF/ID` and `ID/EX`), and freezes all pipeline stages during Instruction Cache misses.
- **Direct-Mapped Instruction Cache (`icache.v`):** 16 cache lines × 4 words/block (64 words = 256 bytes), backed by a 512 × 32-bit synchronous **Block RAM** main instruction memory (`instr_bram.v`). On a miss, 4 words are fetched sequentially from BRAM without burst transfers.
- **Unaligned Data Memory (`data_mem.v`):** 2048-byte memory organized as 4 × 512 × 8-bit Distributed RAM banks supporting single-cycle unaligned byte, half-word, and word transfers.

2 Datapath Block Schematic

Figure 1 illustrates the 5-stage pipelined RV32I datapath, Instruction Cache hierarchy, forwarding paths, and hazard control signals.

Table 1: Direct-Mapped Instruction Cache Address Decomposition (32-bit Byte Address)

Bits [31:8] (24 bits)	**Bits [7:4] (4 bits)**	**Bits [3:2] (2 bits)**	**Bits [1:0] (2 bits)**
Tag	Cache Index (0–15)	Word Offset (0–3)	Byte Offset (`00`)

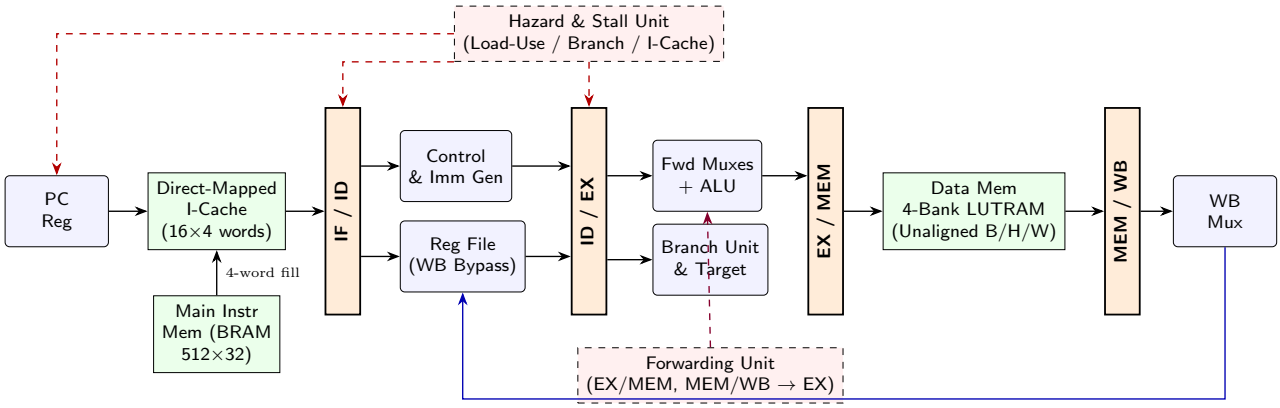


Figure 1: 5-Stage Pipelined RV32I Processor with Direct-Mapped Instruction Cache.

RTL schematic (Vivado elaborated design)

Figure 2 is the schematic Vivado 2026.1 generates after elaboration (*RTL Analysis* → *Open Elaborated Design* → *Schematic*). The top level `basys3_top` contains the processor `u_cpu` (`rv32i_pipeline_cpu`), the seven-segment driver `u_seg7`, and a 4:1 output multiplexer (`disp_val_i`) that selects which 16-bit value drives the LEDs from `sw[1:0]` (quotient, remainder, `pc_out`, status flags). Figure 3 shows the same design with `u_cpu` expanded (711 cells, 1,704 nets).

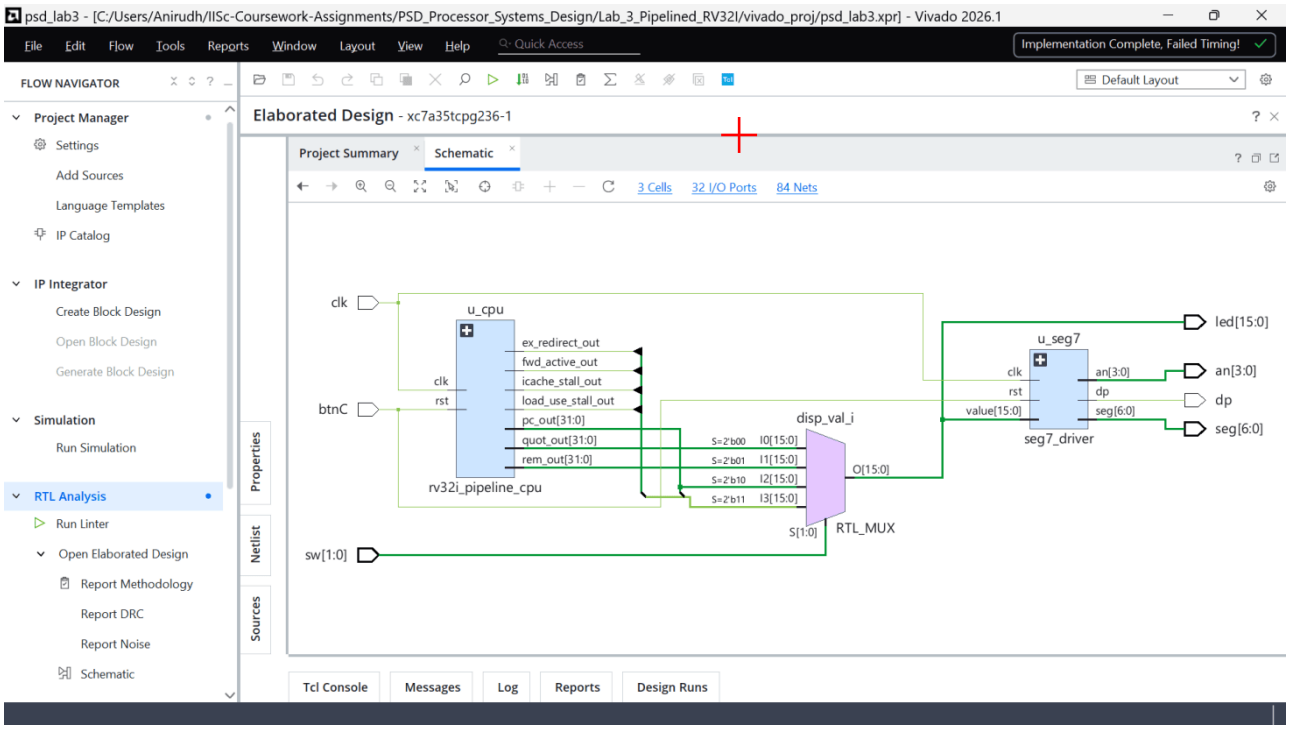


Figure 2: Vivado RTL schematic, top level: `u_cpu`, `u_seg7` and the `sw[1:0]`-controlled output multiplexer.

3 Controller & I-Cache FSM State Diagram

Figure 4 shows the Instruction Cache miss/fill controller FSM, and Table 2 summarizes the pipeline stall and flush truth table.

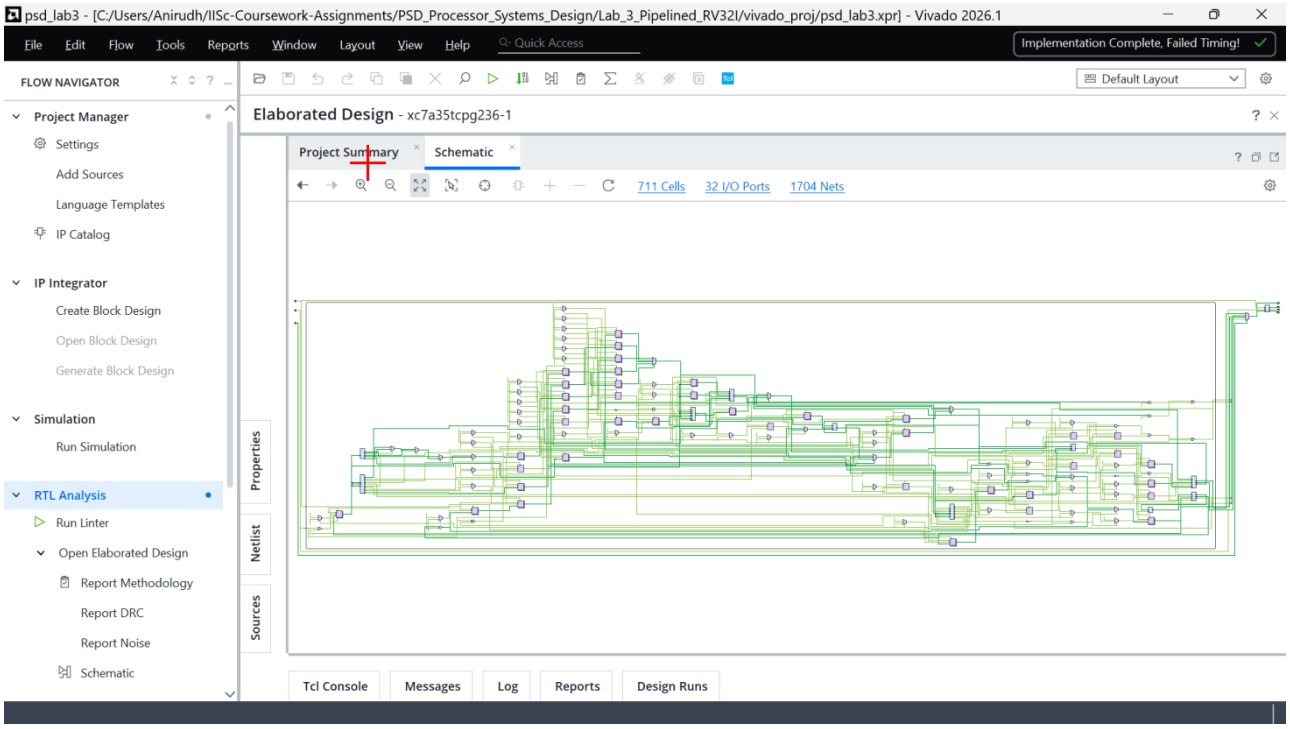


Figure 3: Vivado RTL schematic with `u_cpu` expanded (711 cells, 1,704 nets).

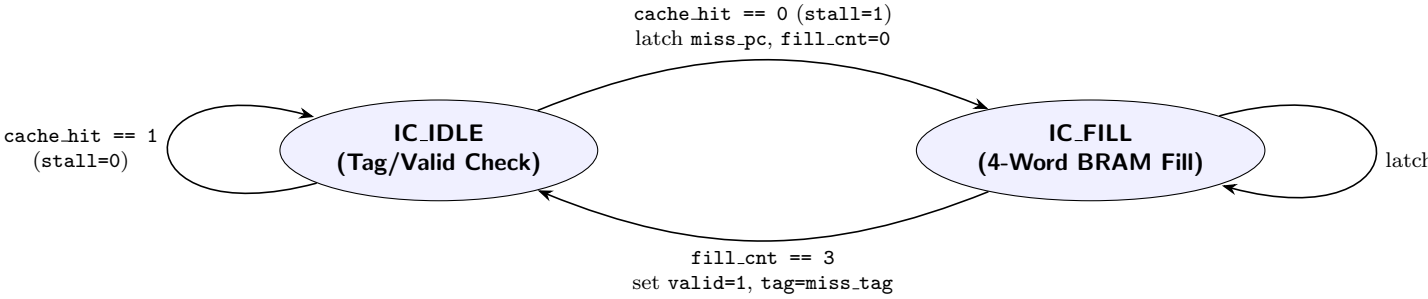


Figure 4: Instruction Cache Miss/Fill Controller FSM.

Table 2: Hazard Detection & Pipeline Stall/Flush Control Truth Table

Condition	`stall_if/id`	`stall_ex/mem/wb`	`flush_if_id`	`flush_id_ex`
`icache_stall == 1`	1	1	0	0
`ex_redirect == 1` (Branch/JAL/JALR)	0	0	1	1
`load_use_stall == 1`	1	0	0	1
Normal Execution	0	0	0	0

4 Verification: 16-bit Unsigned Restoring Division

The processor was tested with a C/RISC-V assembly program (`restoring_div.c`, `restoring_div.asm`) implementing `restoring_divide(100, 7, &rem)` as a function called via `jal ra, restoring_divide` and returning via `jalr x0, ra, 0`, followed by unaligned byte/half-word/word memory tests and a back-to-back load-use hazard.

Table 3: Behavioral (`iverilog`) and Post-Route Timing (`xsim`) Verification Results

Check / Metric	**Observed Value**	**Status**
Quotient `MEM[0x200]` (100/7)	`14` (`0x0000000E`)	PASS (Behavioral & Post-Route)
Remainder `MEM[0x204]` (100 mod 7)	`2` (`0x00000002`)	PASS (Behavioral & Post-Route)
Unaligned `MEM[0x208]` (`sb`/`sh`)	`0xC0DE5A00`	PASS
Unaligned + Load-Use `MEM[0x210]`	`0x00C0DE5B`	PASS
I-Cache Misses / Stall Cycles	10 misses / 50 stall cycles	Verified (5 cycles/miss)
Load-Use Stalls / Branch Flushes / Fwd	1 stall / 106 flushes / 73 fwd	Verified

Behavioral simulation waveforms (Vivado Simulator)

The testbench `rv32i_pipeline_tb` runs 500 clock cycles at 100 MHz, counts I-cache misses, stall cycles, flushes and forwarding events, checks the stored results, and prints the summary of Figure 5.

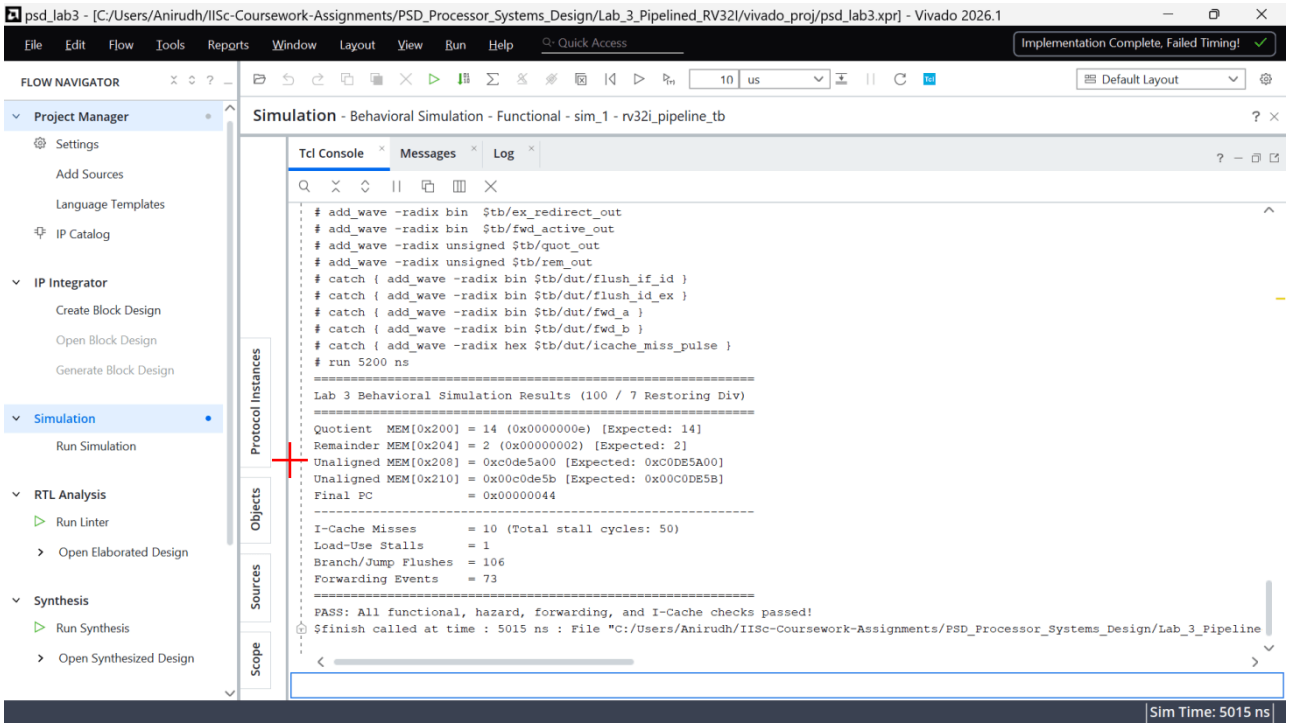


Figure 5: Vivado Tcl console after behavioral simulation: quotient 14, remainder 2, both unaligned-access checks pass, 10 I-cache misses (50 stall cycles), 1 load-use stall, 106 branch/jump flushes, 73 forwarding events; simulation ends at 5,015 ns.

Post-route timing simulation (with SDF back-annotation)

The same program was run on the post-implementation netlist with the SDF file written by Vivado (*Run Simulation → Run Post-Implementation Timing Simulation*), using the top-level testbench `basys3_top_tb`: reset is released after 100 ns (GSR), the program runs for 5,000 ns, then `sw=00` selects the quotient and `sw=01` the remainder on the LEDs.

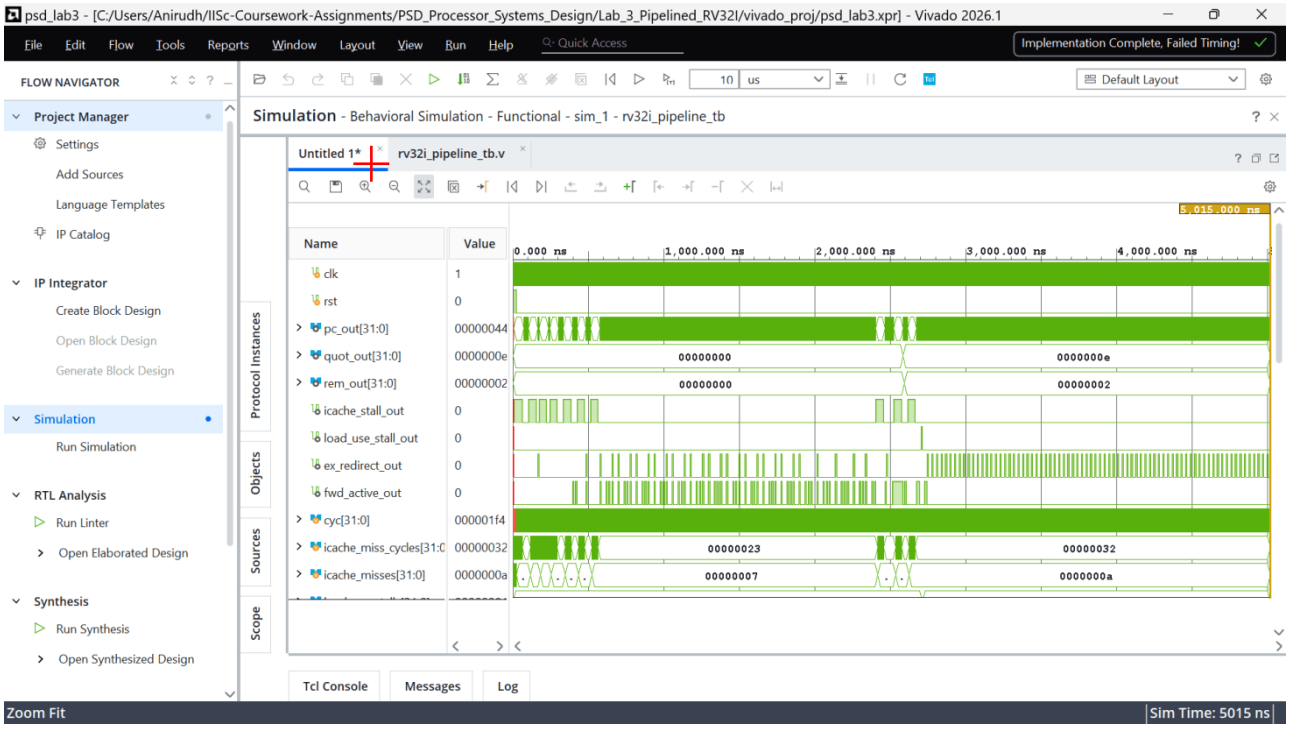


Figure 6: Behavioral simulation, full run (0–5,015 ns). `icache_stall_out` pulses are the I-cache misses (cold start and a second burst near 2.6 µs); `ex_redirect_out` and `fwd_active_out` show branch/jump flushes and forwarding; `quot_out` and `rem_out` become `0x0e` and `0x02` by about 2.6 µs.

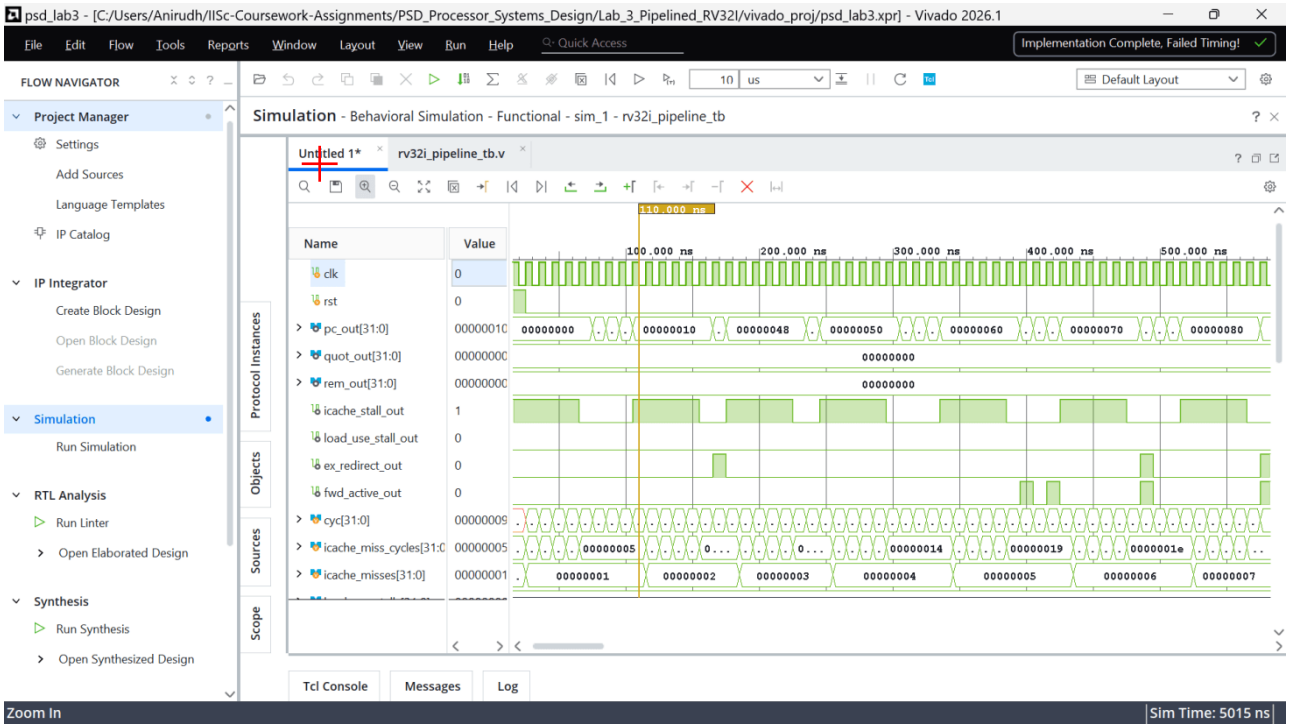


Figure 7: Instruction-cache misses and block fills (0–550 ns). Each miss asserts `icache_stall_out` for five cycles (50 ns) while `pc_out` is frozen; `icache_misses` counts 1 to 7 and `icache_miss_cycles` steps in units of five (`0x05`, `0x14`, `0x19`, `0x1e`). `ex_redirect_out` pulses (branch/jump flush) appear near 170 ns and 490 ns, and `fwd_active_out` pulses near 400–430 ns.

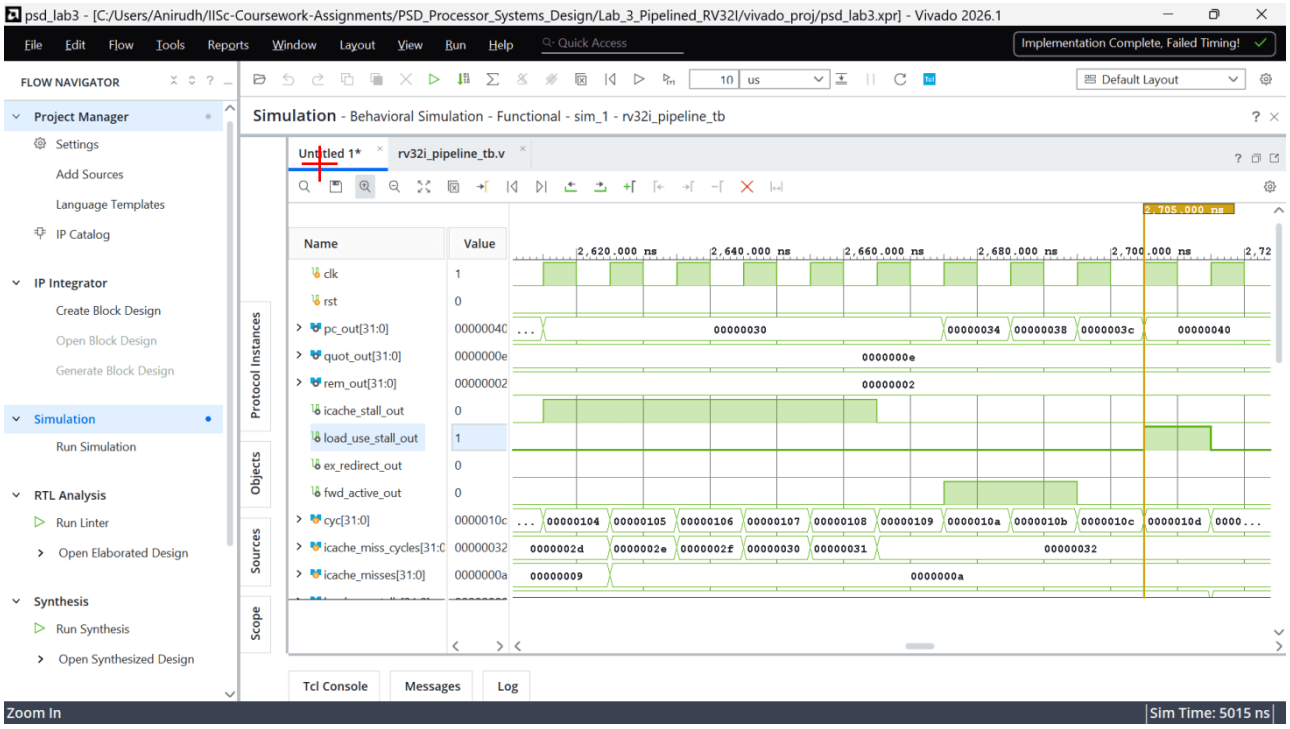


Figure 8: Cache miss, forwarding and the load-use stall (about 2,610–2,730 ns). A five-cycle I-cache stall (about 2,615–2,665 ns) holds `pc_out` at `0x30`; `fwd_active_out` marks a forwarding event (about 2,675–2,695 ns); `load_use_stall_out` is high for one cycle at 2,705–2,715 ns (cursor), the single load-use stall in this program.

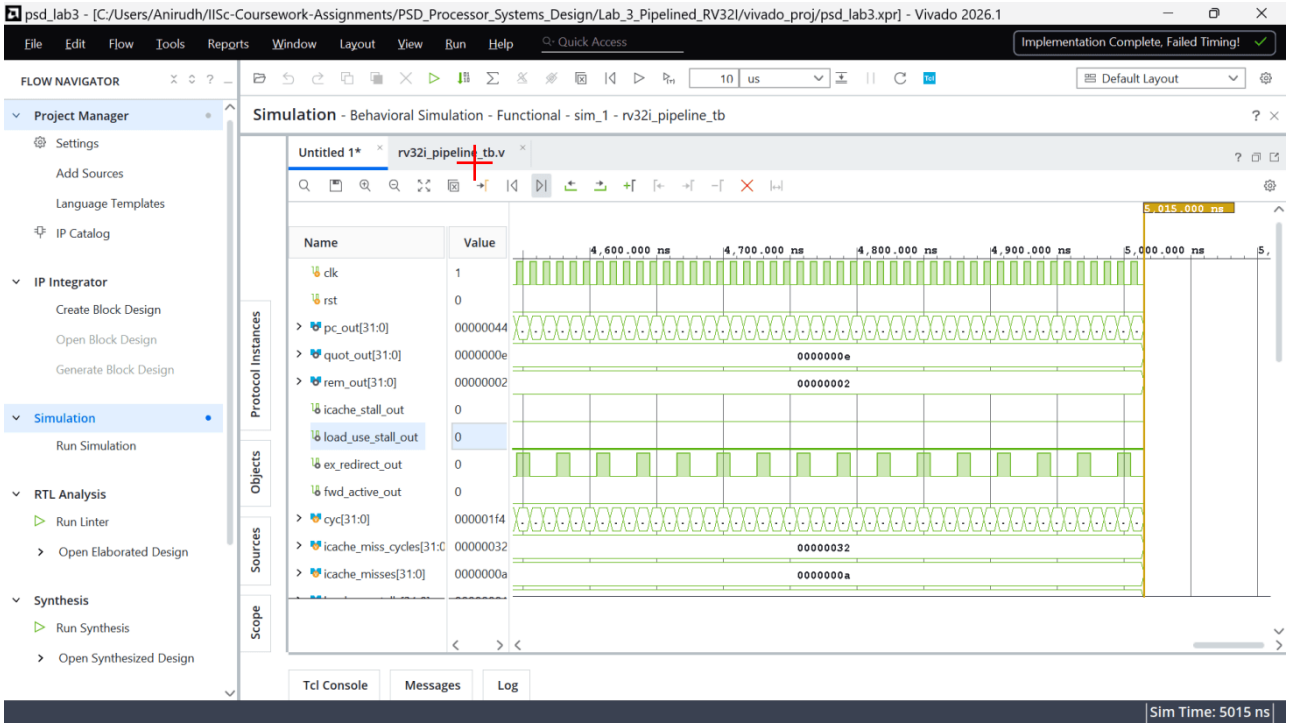


Figure 9: End of the behavioral run (about 4.6–5.0 µs): `quot_out` = `0x0e` (14) and `rem_out` = `0x02` (2) are stable while the program idles in its final loop (`ex_redirect_out` pulses on every jump).

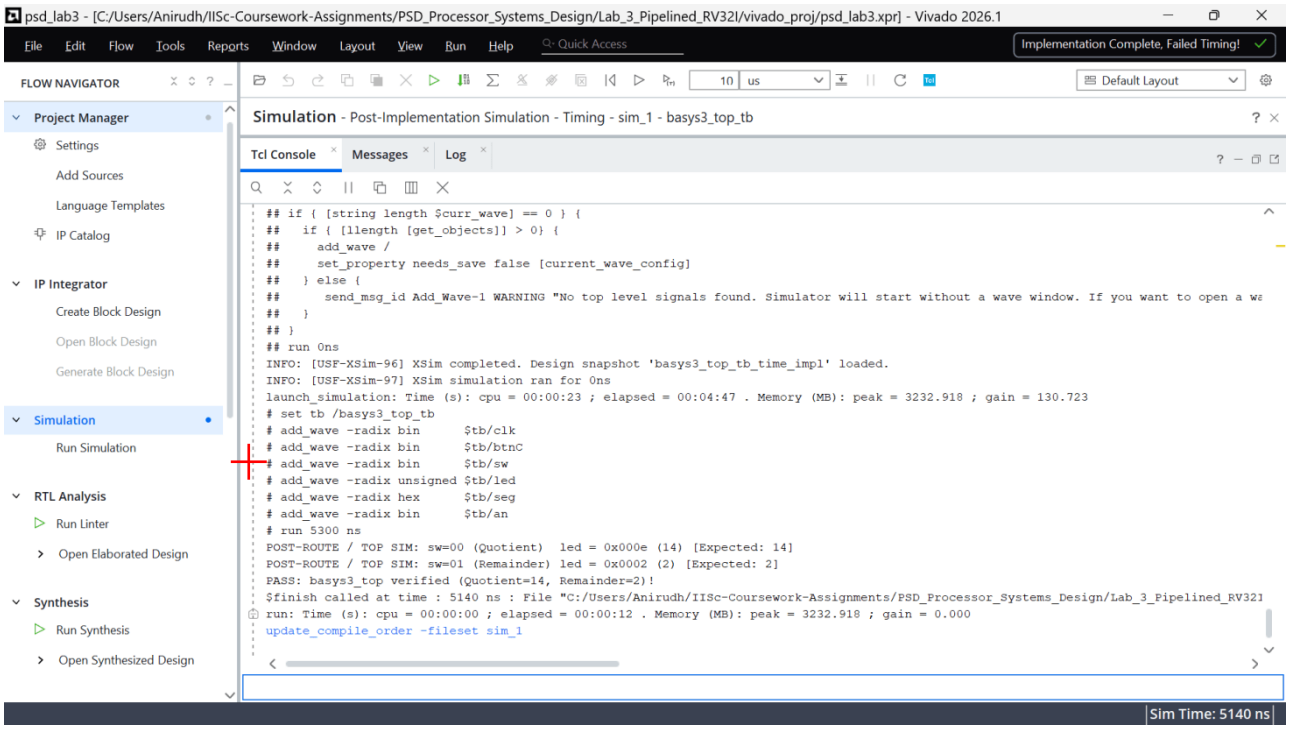


Figure 10: Post-route timing simulation log: `sw=00` gives `led` = `0x000e` (14), `sw=01` gives `led` = `0x0002` (2); "PASS: basys3_top verified"; simulation ends at 5,140 ns.

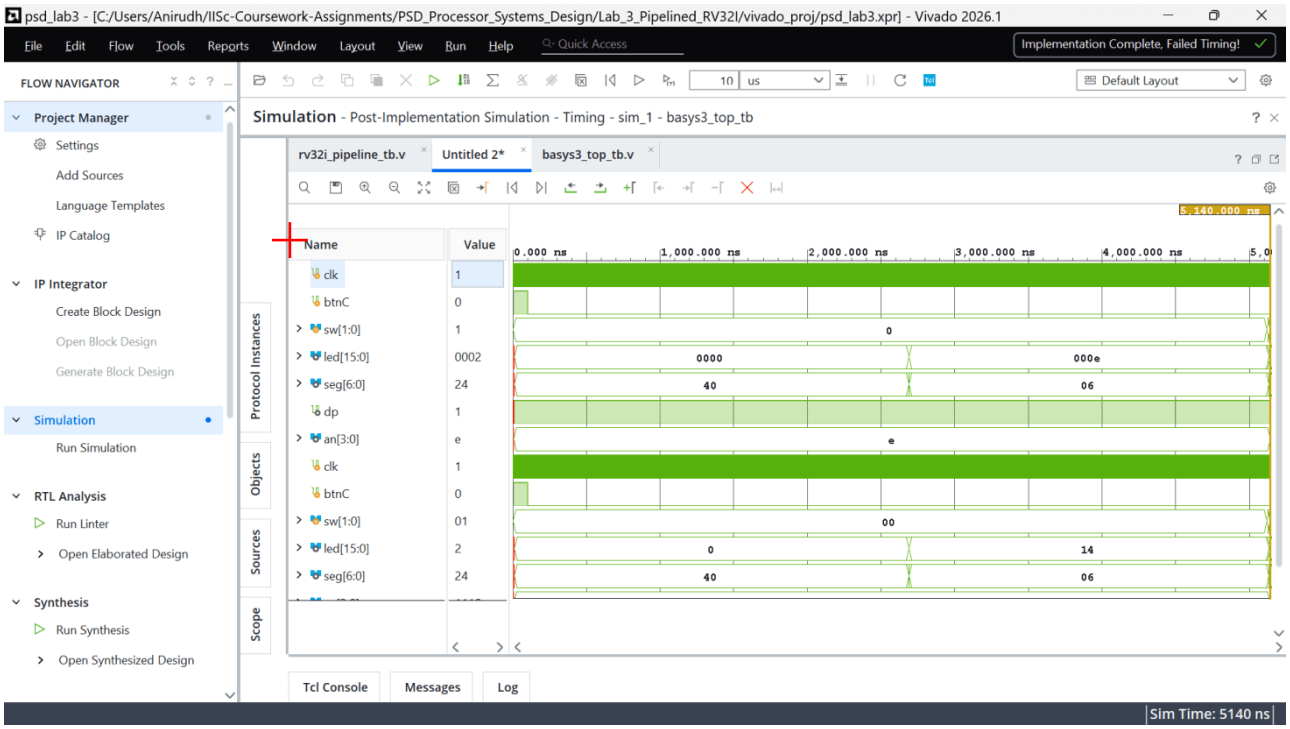


Figure 11: Post-route timing simulation, full run (0–5,140 ns). `led` (shown in hex above, in decimal below) changes from 0 to `0x000e` once the program has stored the quotient (about 2.7 µs) while `sw=00`.

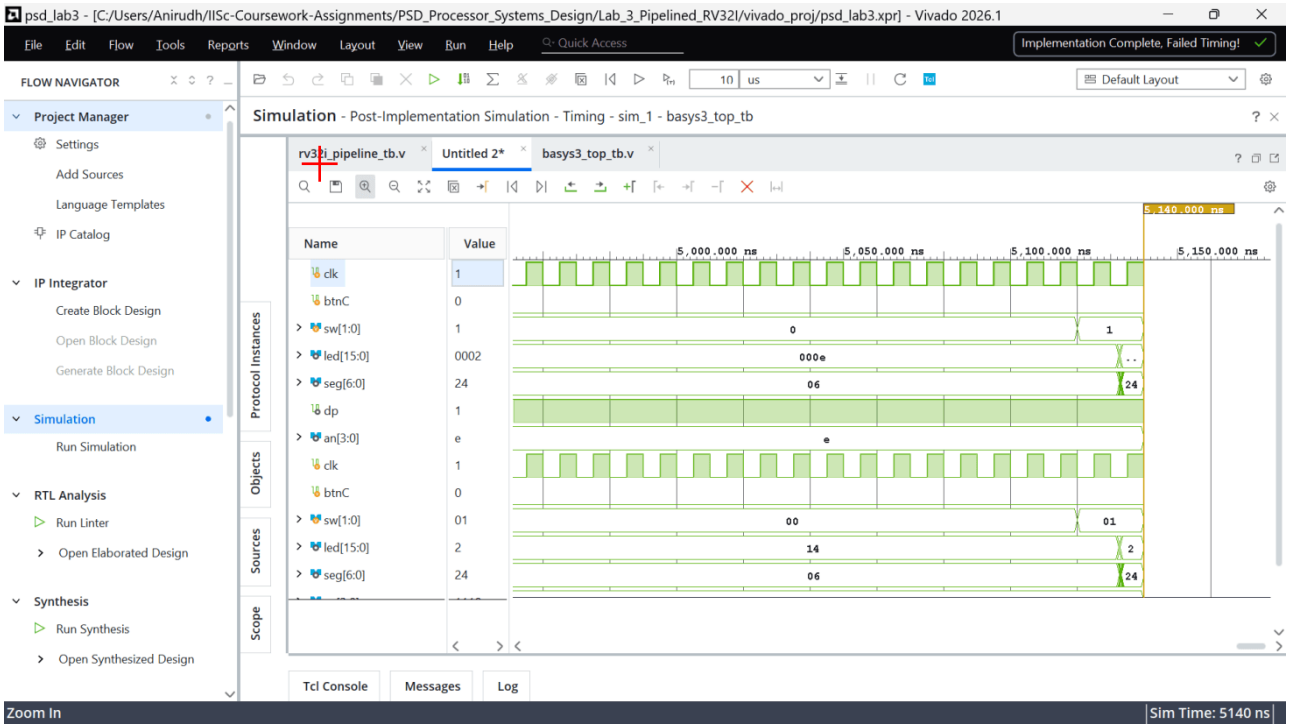


Figure 12: Post-route timing simulation, 5,000–5,140 ns. At about 5,120 ns `sw` changes from `00` to `01`; `led` changes from `0x000e` (14) to `0x0002` (2) after a short unknown interval caused by the back-annotated gate and routing delays, and `seg` changes from `06` to `24`.

5 Vivado 2026.1 Post-Route Timing & Resource Utilization

The complete design (`basys3_top.v`) was synthesized, placed, routed, and verified with post-implementation timing simulation in **Vivado 2026.1** for `xc7a35tcpg236-1` with a 100 MHz (`10.000 ns`) clock constraint.

Table 4: Post-Route Resource Utilization and Timing Summary (`xc7a35tcpg236-1`)

Resource / Timing Metric	Used / Value	Available / Target	Utilization / Margin
Slice LUTs (Total)	2,955	20,800	14.21%
LUT as Logic	2,699	20,800	12.98%
LUT as Distributed RAM (`data_mem`)	256	9,600	2.67%
Slice Registers (Flip-Flops)	4,165	41,600	10.01%
Block RAM (`RAMB18E1` for `instr_bram`)	0.5 Tile (1 RAMB18)	50 Tiles	1.00%
DSPs	0	90	0.00%
Worst Negative Slack (WNS @ 100 MHz)	-0.596 ns	10.000 ns	$F_{\max} \approx 94.37$ MHz
Worst Hold Slack (WHS)	$+0.128$ ns	0.000 ns	0 hold violations
Worst Pulse Width Slack (WPWS)	$+3.750$ ns	0.000 ns	0 pulse-width violations

Timing analysis

The design does **not** meet the 100 MHz constraint: the worst setup slack is -0.596 ns, so the maximum frequency supported by static timing analysis is about $1/(10.000 + 0.596)$ ns ≈ 94.4 MHz. Hold and pulse-width checks pass.

The worst path (Figure 16 and the full path in `reports/rebuild/post_route_timing.rpt`) starts at the write-back register `mem_wb_rd`, passes through the forwarding select logic into `ex_op_a_fwd`, then through the branch comparator's carry chain (`u_branch`) to `ex_branch_cond`, `ex_redirect` and the load-use/stall logic, and finally fans out (fanout 190) to the synchronous reset of the ID/EX pipeline

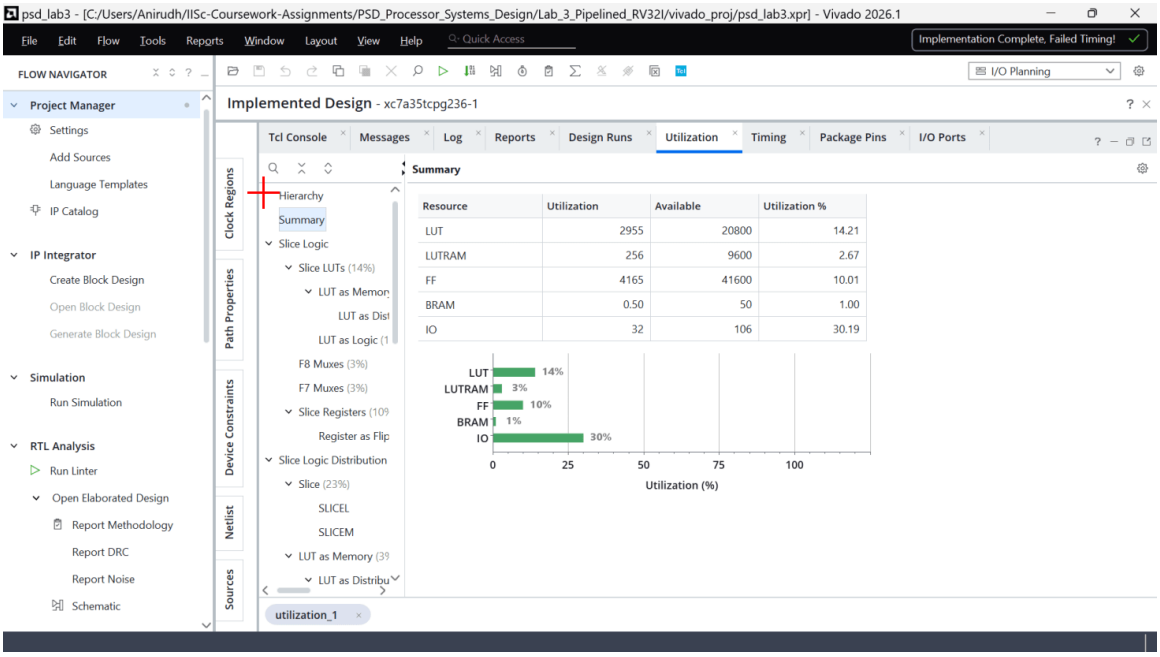


Figure 13: Vivado post-route utilization summary: 2,955 LUTs (14.21%), 256 LUTRAM (2.67%), 4,165 flip-flops (10.01%), 0.5 BRAM tile (1.00%), 32 I/O (30.19%).

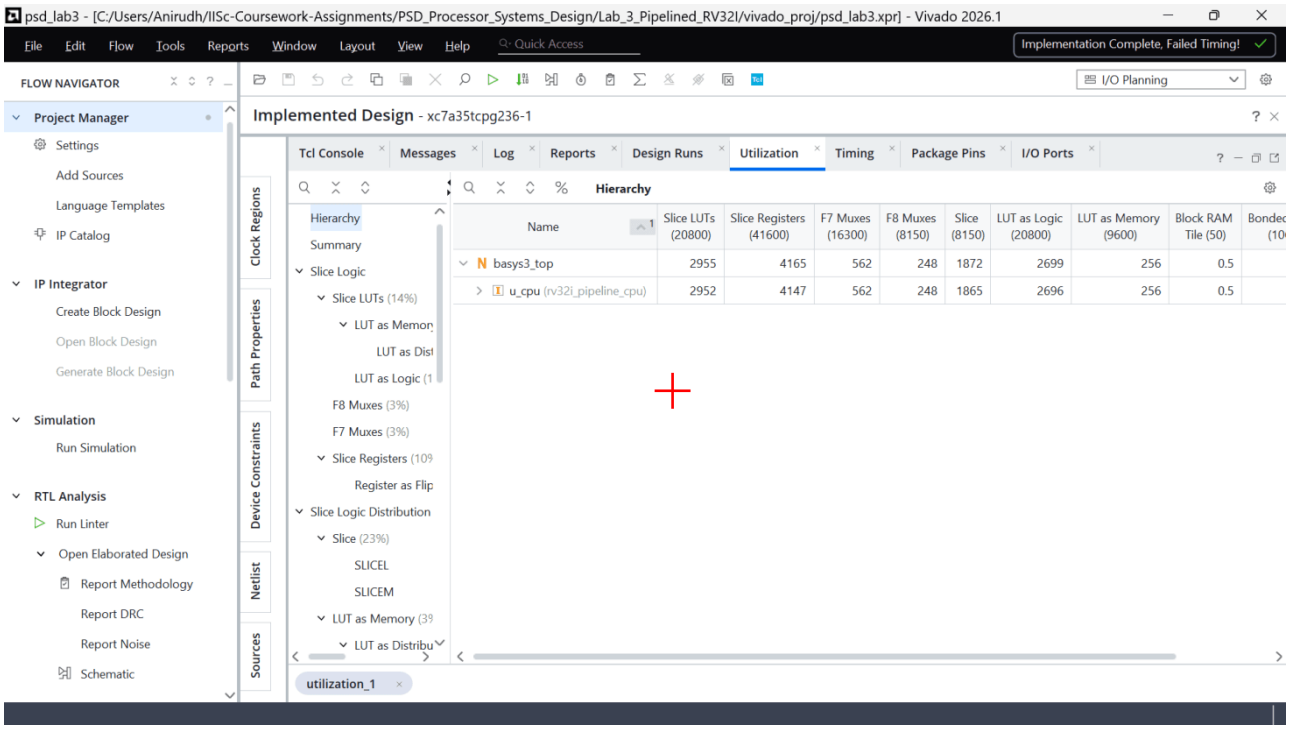


Figure 14: Post-route utilization by hierarchy: `u_cpu` accounts for 2,952 of the 2,955 LUTs and 4,147 of the 4,165 registers.

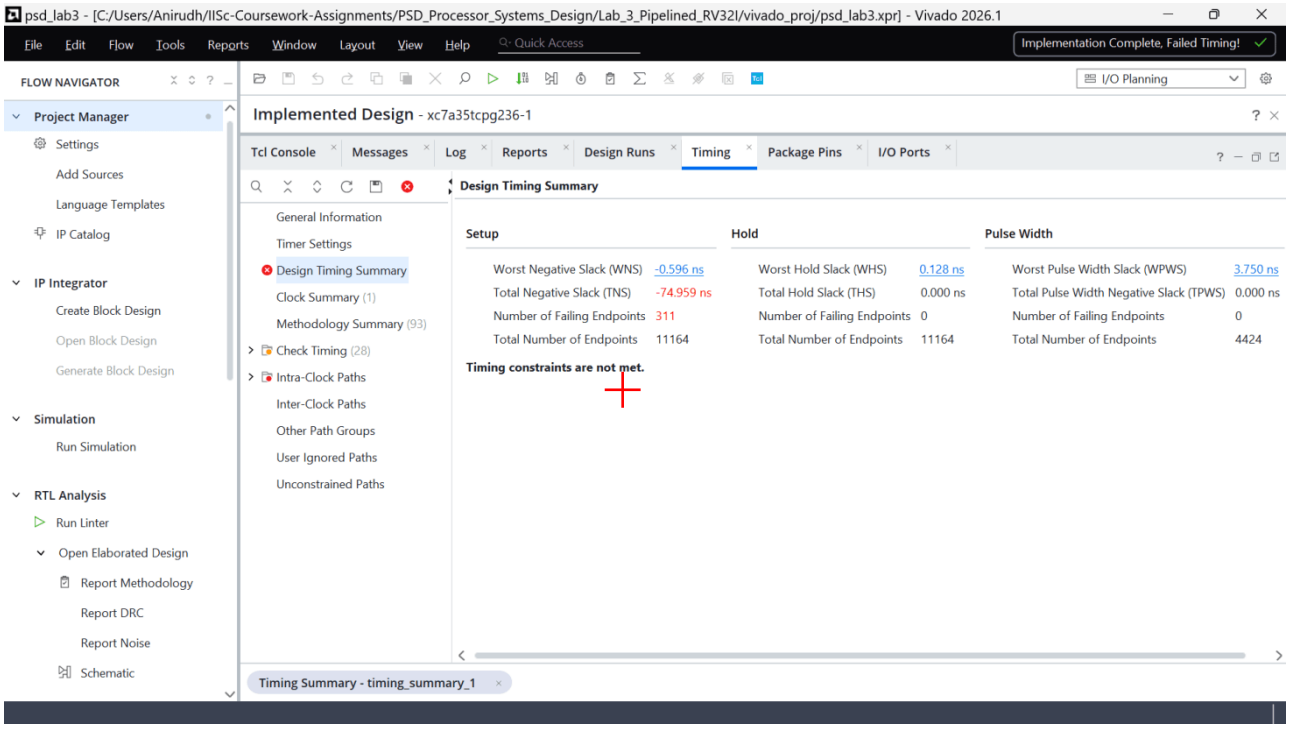


Figure 15: Vivado Design Timing Summary at the 10 ns (100 MHz) constraint: WNS = −0.596 ns, TNS = −74.959 ns with 311 failing setup endpoints out of 11,164; WHS = +0.128 ns (0 failing); WPWS = 3.750 ns. Vivado reports "Timing constraints are not met".

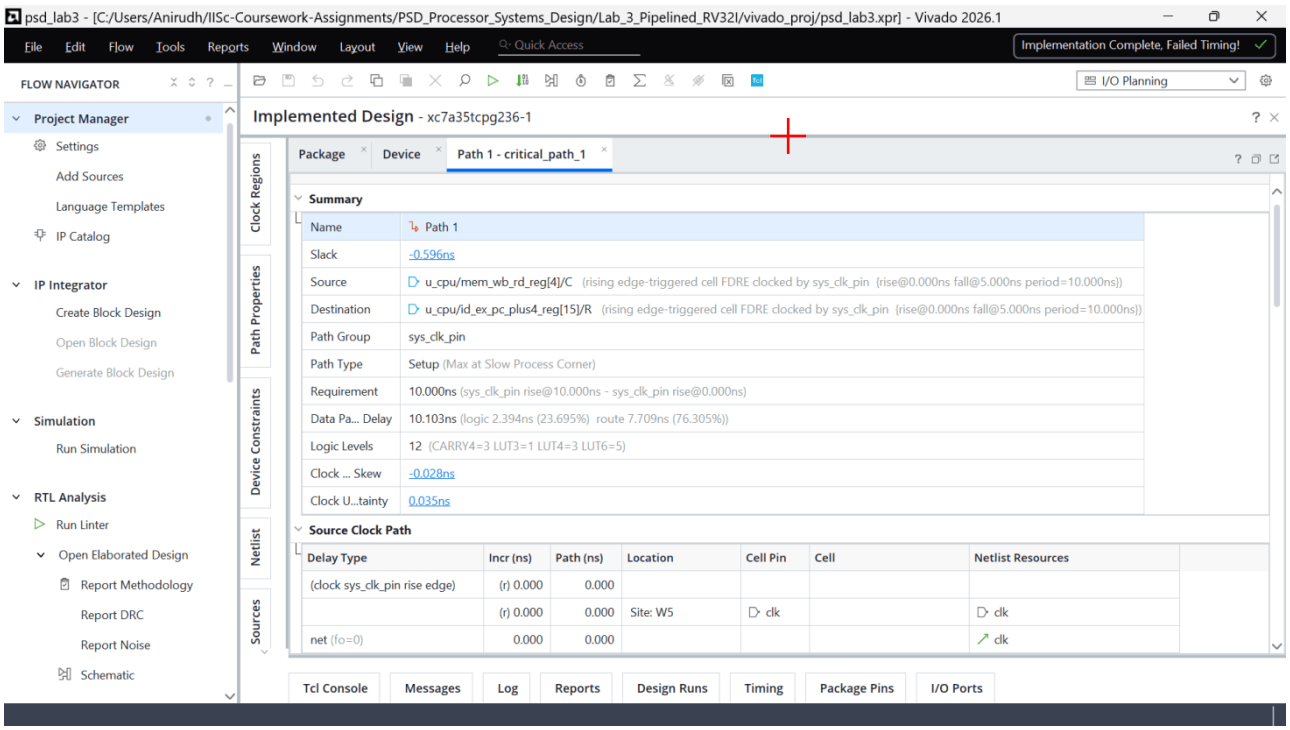


Figure 16: Worst setup path (Slow process corner): from `u_cpu/mem_wb_rd_reg[4]` to the reset pin of `u_cpu/id_ex_pc_plus4_reg[15]`. Data-path delay 10.103 ns, of which logic 2.394 ns (23.7%) and routing 7.709 ns (76.3%); 12 logic levels (CARRY4 ×3, LUT3 ×1, LUT4 ×3, LUT6 ×5).

registers. In other words it is the *forward* → *compare* → *redirect* → *flush* chain inside the EX stage, and 76% of its delay is routing rather than logic.

Ways to shorten it, which were not implemented in this submission, would be to register part of the redirect decision, reduce the fanout of the ID/EX flush signal (replicate it or use clock enables), or simply constrain the clock to about 10.6 ns (94 MHz).

The post-route timing simulation in the previous section passes at the 10 ns clock. That does not contradict the failing slack: the simulation only exercises the paths this particular program sensitises, whereas static timing analysis checks every path at the slow process corner. The correct statement of the achieved performance is therefore the static-timing result above, not 100 MHz.

A C code

```c
/*
 * ============================================================================
 * E3: 245 Processor System Design - IISc DESE
 * Lab Exercise 2: 32-bit Single Cycle RISC-V Processor (RV32I)
 * C Program for 16-bit Unsigned Restoring Division Algorithm
 * ============================================================================
 */

// Function: restoring_divide
// Performs 16-bit restoring division algorithm:
// Inputs:
//   dividend: 16-bit unsigned dividend (e.g. 100)
//   divisor : 16-bit unsigned divisor  (e.g. 7)
// Outputs:
//   Returns quotient (e.g. 14) and writes remainder to *rem_ptr (e.g. 2)
unsigned int restoring_divide(unsigned int dividend, unsigned int divisor, unsigned
    int *rem_ptr) {
    unsigned int Q = dividend & 0xFFFF; // Quotient / Working dividend
    unsigned int D = divisor & 0xFFFF;  // Divisor
    unsigned int R = 0;                 // Remainder accumulator
    int count = 16;                     // 16-bit division loop

    while (count > 0) {
        // Step 1: Shift R left by 1 bit, pull in MSB of current Q (bit 15)
        unsigned int msb_q = (Q >> 15) & 1;
        R = (R << 1) | msb_q;

        // Step 2: Shift Q left by 1 bit
        Q = (Q << 1) & 0xFFFF;

        // Step 3: Trial subtraction: R - D
        if (R >= D) {
            R = R - D;      // Subtraction successful, keep subtracted remainder
            Q = Q | 1;      // Set quotient bit to 1
        } else {
            // Trial subtraction negative -> Restore R (remains unchanged)
            // Quotient bit remains 0
        }

        count--;
    }

    if (rem_ptr) {
        *rem_ptr = R;
    }
    return Q;
}

// Main Program
int main(void) {
```

```
    // Test Vector: 100 / 7
    unsigned int dividend = 100;
    unsigned int divisor  = 7;
    unsigned int rem = 0;

    // Call restoring division function
    unsigned int quot = restoring_divide(dividend, divisor, &rem);

    // Write results to designated data memory locations:
    // Address 0x200 (Word 128) = Quotient  (Expected: 14 = 0x000E)
    // Address 0x204 (Word 129) = Remainder (Expected: 2  = 0x0002)
    volatile unsigned int *quot_mem = (volatile unsigned int *)0x200;
    volatile unsigned int *rem_mem  = (volatile unsigned int *)0x204;
    *quot_mem = quot;
    *rem_mem  = rem;

    // Unaligned transfer validation:
    // Test byte and half-word unaligned store and load
    volatile unsigned char  *unaligned_byte = (volatile unsigned char *)0x209;
    volatile unsigned short *unaligned_half = (volatile unsigned short *)0x20A;
    *unaligned_byte = 0x5A;
    *unaligned_half = 0xC0DE;

    // Halt in infinite loop
    while (1);

    return 0;
}
```

B Assembly code

```
# ================================================================================
# E3: 245 Processor System Design - IISc DESE
# Laboratory Exercise 2: 32-bit Single Cycle RISC-V Processor (RV32I)
# Assembly Program: 16-bit Unsigned Restoring Division Algorithm
# ================================================================================

.org 0x0000

main:
    # Initialize Stack Pointer (top of memory space, word 448 = 0x700)
    li   sp, 0x700

    # Test Vector: Dividend = 100 (0x0064), Divisor = 7 (0x0007)
    li   a0, 100          # a0 = Dividend
    li   a1, 7            # a1 = Divisor

    # Call restoring_divide function (Passes a0, a1; returns a0=Quot, a1=Rem)
    jal  ra, restoring_divide

    # Save results to Data Memory:
    # Address 0x200 (Word 128) = Quotient  (Expected: 14 = 0x000E)
    # Address 0x204 (Word 129) = Remainder (Expected: 2  = 0x0002)
    li   t0, 0x200
    sw   a0, 0(t0)        # MEM[0x200] = Quotient
    sw   a1, 4(t0)        # MEM[0x204] = Remainder

    # ================================================================================
    # Verification of Unaligned Word, Half-Word, and Byte Transfers
    # ================================================================================
    li   t2, 0x5A
    sb   t2, 9(t0)        # Unaligned byte store at 0x209
    lbu  t4, 9(t0)        # Unaligned byte load from 0x209 (Expect 0x5A)
```

```
    li   t3, 0xC0DE
    sh   t3, 10(t0)       # Unaligned half-word store at 0x20A
    lhu  t5, 10(t0)       # Unaligned half-word load from 0x20A (Expect 0xC0DE)

    # Unaligned word load across boundary (at 0x209: bytes 0x209, 0x20A, 0x20B, 0
        x20C)
    lw   t6, 9(t0)        # Reads 32-bit unaligned word
    addi t6, t6, 1        # Load-use hazard: immediate use of t6 after lw
    sw   t6, 16(t0)       # Store incremented value at 0x210

halt:
    j    halt             # Infinite loop terminating test program

# ================================================================================
# Function: restoring_divide
# Algorithm: 16-bit Unsigned Restoring Division
# Input:
#   a0 = 16-bit Dividend
#   a1 = 16-bit Divisor
# Output:
#   a0 = 16-bit Quotient  (14 = 0x000E)
#   a1 = 16-bit Remainder (2  = 0x0002)
# Register Usage:
#   t0 = Q (working dividend / quotient register)
#   t1 = D (divisor)
#   t2 = R (remainder register, initialized to 0)
#   t3 = Loop Counter (16 downto 1)
#   t4 = Shift bit extracted from Q
# ================================================================================
restoring_divide:
    mv   t0, a0           # t0 = Q = Dividend
    mv   t1, a1           # t1 = D = Divisor
    li   t2, 0            # t2 = R = 0 (Remainder accumulator)
    li   t3, 16           # t3 = 16 (Loop iteration counter)

div_loop:
    # 1. Extract MSB of current Q (bit 15)
    srli t4, t0, 15
    andi t4, t4, 1

    # 2. Shift R left by 1 and pull in MSB of Q
    slli t2, t2, 1
    or   t2, t2, t4

    # 3. Shift Q left by 1
    slli t0, t0, 1

    # 4. Trial subtraction: R - D
    # If R < D, trial subtraction borrows -> Restore R (keep unchanged), Q LSB = 0
    # If R >= D, subtract D from R, set Q LSB = 1
    bltu t2, t1, no_sub
    sub  t2, t2, t1       # Subtraction successful: R = R - D
    ori  t0, t0, 1        # Set quotient bit: Q = Q | 1

no_sub:
    # Loop control: count down from 16 to 0
    addi t3, t3, -1
    bne  t3, x0, div_loop

    # Return results in a0 (Quotient) and a1 (Remainder)
    slli a0, t0, 16
    srli a0, a0, 16       # a0 = Quotient (14 = 0x000E)
    slli a1, t2, 16
    srli a1, a1, 16       # a1 = Remainder (2  = 0x0002)
    ret                   # Return to caller: jalr x0, ra, 0
```

```
# ================================================================================
# Data Section (Memory initialized to 0)
# ================================================================================
.org 0x0200
DATA_QUOTIENT:
    .word 0x00000000
DATA_REMAINDER:
    .word 0x00000000
DATA_UNALIGNED:
    .word 0x00000000
    .word 0x00000000
```

C Constraint file (basys3.xdc)

```
# ================================================================================
# E3: 245 Processor System Design - IISc DESE
# Constraints File for Digilent Basys 3 FPGA Board
# Target Device: Xilinx Artix-7 XC7A35T-1CPG236C
# IO Standard: LVCMOS33 on all pins
# ================================================================================

# 1. 100 MHz Master Oscillator (Pin W5)
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [
    get_ports clk]

# 2. Push-Button Center (Reset, Pin U18)
set_property PACKAGE_PIN U18 [get_ports btnC]
set_property IOSTANDARD LVCMOS33 [get_ports btnC]

# 3. Slide Switches (sw[0] = V17, sw[1] = V16)
set_property PACKAGE_PIN V17 [get_ports {sw[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
set_property PACKAGE_PIN V16 [get_ports {sw[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]

# 4. 16 Discrete LEDs (Pins U16 down to L1)
set_property PACKAGE_PIN U16 [get_ports {led[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
set_property PACKAGE_PIN E19 [get_ports {led[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
set_property PACKAGE_PIN U19 [get_ports {led[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
set_property PACKAGE_PIN V19 [get_ports {led[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
set_property PACKAGE_PIN W18 [get_ports {led[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
set_property PACKAGE_PIN U15 [get_ports {led[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
set_property PACKAGE_PIN U14 [get_ports {led[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
set_property PACKAGE_PIN V14 [get_ports {led[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
set_property PACKAGE_PIN V13 [get_ports {led[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[8]}]
set_property PACKAGE_PIN V3  [get_ports {led[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[9]}]
set_property PACKAGE_PIN W3  [get_ports {led[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[10]}]
set_property PACKAGE_PIN U3  [get_ports {led[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[11]}]
set_property PACKAGE_PIN P3  [get_ports {led[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[12]}]
```

```
set_property PACKAGE_PIN N3  [get_ports {led[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[13]}]
set_property PACKAGE_PIN P1  [get_ports {led[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[14]}]
set_property PACKAGE_PIN L1  [get_ports {led[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[15]}]

# 5. Seven Segment Display Cathodes (Pins W7..U7: a, b, c, d, e, f, g)
set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]

# 6. Seven Segment Display Decimal Point (Pin V7)
set_property PACKAGE_PIN V7 [get_ports dp]
set_property IOSTANDARD LVCMOS33 [get_ports dp]

# 7. Seven Segment Display Anodes (Pins U2, U4, V4, W4: an[0]..an[3])
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]

# 8. FPGA Configuration Constraints
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

D Key Verilog sources

The complete source files are in the `rtl/` folder: `rv32i_pipeline_cpu.v`, `icache.v`, `instr_bram.v`, `hazard_unit.v`, `forwarding_unit.v`, `branch_unit.v`, `alu.v`, `control_unit.v`, `imm_gen.v`, `reg_file.v`, `data_mem.v`, `seg7_driver.v`, `basys3_top.v`, `rv32i_defines.v`. The testbenches are in `testbench/`. The three modules that implement the stall, forwarding and cache behaviour shown in the waveforms are listed below.

`hazard_unit.v`

```
// ============================================================================
// E3: 245 Processor System Design - IISc DESE
// Lab Exercise 3: 5-Stage Pipelined RISC-V Processor with Instruction Cache
// Hazard Detection & Stall/Flush Control Unit
// ============================================================================

module hazard_unit (
    input  wire       icache_stall,
    input  wire       ex_redirect,      // Taken branch, JAL, or JALR in EX
    input  wire       id_ex_mem_read,   // Load instruction in EX
    input  wire [4:0] id_ex_rd,
```

```
    input  wire [4:0] if_id_rs1,
    input  wire [4:0] if_id_rs2,
    output wire       stall_if,
    output wire       stall_id,
    output wire       stall_ex,
    output wire       stall_mem,
    output wire       stall_wb,
    output wire       flush_if_id,
    output wire       flush_id_ex,
    output wire       load_use_stall
);

    assign load_use_stall = id_ex_mem_read && (id_ex_rd != 5'd0) &&
                            ((id_ex_rd == if_id_rs1) || (id_ex_rd == if_id_rs2)) &&
                            !ex_redirect;

    // I-Cache miss stalls the entire pipeline cleanly
    assign stall_if  = icache_stall || load_use_stall;
    assign stall_id  = icache_stall || load_use_stall;
    assign stall_ex  = icache_stall;
    assign stall_mem = icache_stall;
    assign stall_wb  = icache_stall;

    // Flush wrong-path instructions on branch/jump redirect, or insert bubble on
       load-use
    assign flush_if_id = !icache_stall && ex_redirect;
    assign flush_id_ex = !icache_stall && (ex_redirect || load_use_stall);

endmodule
```

forwarding_unit.v

```
// ============================================================================
// E3: 245 Processor System Design - IISc DESE
// Lab Exercise 3: 5-Stage Pipelined RISC-V Processor with Instruction Cache
// Data Forwarding Unit (EX/MEM -> EX and MEM/WB -> EX)
// ============================================================================

module forwarding_unit (
    input  wire [4:0] id_ex_rs1,
    input  wire [4:0] id_ex_rs2,
    input  wire [4:0] ex_mem_rd,
    input  wire       ex_mem_reg_write,
    input  wire [4:0] mem_wb_rd,
    input  wire       mem_wb_reg_write,
    output reg  [1:0] fwd_a,
    output reg  [1:0] fwd_b
);

    // 2'b00 = ID/EX register operand
    // 2'b10 = Forward from EX/MEM stage output (priority)
    // 2'b01 = Forward from MEM/WB stage output
    always @(*) begin
        if (ex_mem_reg_write && (ex_mem_rd != 5'd0) && (ex_mem_rd == id_ex_rs1))
            fwd_a = 2'b10;
        else if (mem_wb_reg_write && (mem_wb_rd != 5'd0) && (mem_wb_rd == id_ex_rs1)
            )
            fwd_a = 2'b01;
        else
            fwd_a = 2'b00;

        if (ex_mem_reg_write && (ex_mem_rd != 5'd0) && (ex_mem_rd == id_ex_rs2))
            fwd_b = 2'b10;
```

```
        else if (mem_wb_reg_write && (mem_wb_rd != 5'd0) && (mem_wb_rd == id_ex_rs2)
            )
            fwd_b = 2'b01;
        else
            fwd_b = 2'b00;
    end

endmodule
```

icache.v

```
// ============================================================================
// E3: 245 Processor System Design - IISc DESE
// Lab Exercise 3: 5-Stage Pipelined RISC-V Processor with Instruction Cache
// Direct-Mapped Instruction Cache (16 lines x 4 words/block = 64 words)
// Word-by-word sequential fill from Block RAM on cache miss
// ============================================================================

module icache (
    input  wire        clk,
    input  wire        rst,
    input  wire [31:0] pc,
    output wire [31:0] instr,
    output wire        stall,
    output wire        miss_pulse,
    // Interface to Main Instruction Memory (Block RAM)
    output reg  [8:0]  bram_addr,
    input  wire [31:0] bram_rdata
);

    localparam NUM_LINES = 16;
    localparam IC_IDLE   = 1'b0;
    localparam IC_FILL   = 1'b1;

    reg        state;
    reg [31:0] miss_pc;
    reg [1:0]  fill_cnt;

    // Cache storage: 16 lines, 4 words per line
    reg        valid_arr [0:NUM_LINES-1];
    reg [23:0] tag_arr   [0:NUM_LINES-1];
    reg [31:0] data_arr  [0:NUM_LINES-1][0:3];

    // Address decomposition for current PC
    wire [1:0]  word_off = pc[3:2];
    wire [3:0]  index    = pc[7:4];
    wire [23:0] tag      = pc[31:8];

    // Address decomposition for missed PC
    wire [3:0]  miss_idx = miss_pc[7:4];
    wire [23:0] miss_tag = miss_pc[31:8];

    wire cache_hit = (state == IC_IDLE) && valid_arr[index] && (tag_arr[index] ==
        tag);

    assign instr      = cache_hit ? data_arr[index][word_off] : 32'h00000013; // NOP
         on miss
    assign stall      = !cache_hit;
    assign miss_pulse = (state == IC_IDLE) && !cache_hit;

    // Next BRAM address:
    // In IC_IDLE on miss, request word 0 of the 4-word block.
    // In IC_FILL, request word (fill_cnt + 1) while latching word (fill_cnt).
    wire [1:0] next_word = fill_cnt + 2'd1;
```

```
    always @(*) begin
        if (state == IC_IDLE)
            bram_addr = {pc[10:4], 2'b00};
        else
            bram_addr = {miss_pc[10:4], next_word};
    end

    integer i, w;
    always @(posedge clk) begin
        if (rst) begin
            state    <= IC_IDLE;
            miss_pc  <= 32'd0;
            fill_cnt <= 2'd0;
            for (i = 0; i < NUM_LINES; i = i + 1) begin
                valid_arr[i] <= 1'b0;
                tag_arr[i]   <= 24'd0;
                for (w = 0; w < 4; w = w + 1)
                    data_arr[i][w] <= 32'h00000013;
            end
        end else begin
            case (state)
                IC_IDLE: begin
                    if (!cache_hit) begin
                        miss_pc  <= pc;
                        fill_cnt <= 2'd0;
                        state    <= IC_FILL;
                    end
                end

                IC_FILL: begin
                    data_arr[miss_idx][fill_cnt] <= bram_rdata;
                    if (fill_cnt == 2'd3) begin
                        valid_arr[miss_idx] <= 1'b1;
                        tag_arr[miss_idx]   <= miss_tag;
                        state               <= IC_IDLE;
                    end else begin
                        fill_cnt <= fill_cnt + 2'd1;
                    end
                end
            endcase
        end
    end

endmodule
```